

what are the key step to keep in mind while build a project

When building a project (software, business, research, or even a personal project), these are the key steps and principles to keep in mind:

1. Define the Problem Clearly

- What problem are you solving?
- Who are the users or stakeholders?
- Why does this problem matter?

A well-defined problem prevents building features that nobody needs.

2. Set Clear Goals

- Define success criteria.
- Establish measurable outcomes.
- Prioritize must-have vs. nice-to-have features.

Example:

- Must-have: User login.
- Nice-to-have: Social media integration.

3. Plan Before Building

Create:

- Requirements list
- Scope definition
- Timeline and milestones
- Risk assessment

Avoid jumping directly into implementation without a plan.

4. Design the Solution

Think about:

- Architecture
- User experience (UX)
- Data flow

- Scalability
- Security

Simple designs are usually easier to maintain and improve.

5. Build an MVP First

An MVP (Minimum Viable Product) contains only the core functionality needed to validate the idea.

Benefits:

- Faster feedback
- Lower cost
- Reduced risk

6. Use Version Control

For software projects:

- Use Git .
- Store code in repositories such as GitHub , GitLab , or Bitbucket .
- Commit changes frequently with meaningful messages.

7. Test Early and Often

Types of testing:

- Unit testing
- Integration testing
- User acceptance testing
- Performance testing

Finding bugs early is much cheaper than fixing them later.

8. Manage Risks

Ask:

- What could fail?
- What dependencies exist?
- What is the backup plan?

Common risks:

- Scope creep
- Budget overruns
- Technical limitations

- Resource shortages

9. Document Important Decisions

Keep records of:

- Requirements
- Architecture choices
- Setup instructions
- Known issues

Good documentation saves significant time later.

10. Gather Feedback Continuously

Don't wait until the project is finished.

- Show progress regularly.
- Collect user feedback.
- Adjust priorities based on real needs.

11. Monitor Progress

Track:

- Tasks completed
- Deadlines
- Budget
- Quality metrics

Tools such as Jira , Trello , or Asana can help.

12. Prepare for Maintenance

A project isn't finished at launch.

Plan for:

- Bug fixes
- Updates
- Security patches
- User support

A Simple Project Checklist

Before starting, make sure you can answer:

- ✓ What problem am I solving?
- ✓ Who is the target user?
- ✓ What does success look like?
- ✓ What is the MVP?
- ✓ What are the biggest risks?
- ✓ How will I test it?
- ✓ How will I gather feedback?
- ✓ How will I maintain it after launch?

For software development specifically, a practical sequence is:

Requirements → Design → MVP → Development → Testing → Deployment → Feedback → Iteration → Maintenance.



Monday.com
Sponsored



When reporting takes too long

Get clear PMO reports without manual updates.

Done

in realtime software project

In a real-world software project, success often depends less on coding and more on planning, communication, and execution. Here are the most important things to keep in mind:

1. Understand Requirements Thoroughly

Before writing code:

- Understand the business problem.
- Clarify ambiguous requirements.
- Identify edge cases.
- Define acceptance criteria.

A common mistake is starting development before requirements are fully understood.

2. Break Work into Small Tasks

Instead of building everything at once:

- Create user stories or tasks.
- Estimate effort.
- Prioritize features.
- Deliver incrementally.

This makes progress easier to track and reduces risk.

3. Design Before Coding

Think about:

- System architecture
- Database design
- API contracts
- Security requirements
- Scalability needs

Good design prevents major refactoring later.

4. Follow Coding Standards

Maintain:

- Consistent naming conventions
- Clean code structure
- Reusable components
- Proper comments where necessary

Write code that teammates can understand six months later.

5. Use Version Control Properly

With tools like Git :

- Create feature branches.
- Write meaningful commit messages.
- Use pull requests.
- Avoid committing broken code.

Example:

- Good: Add password reset API

- Poor: fixed stuff

6. Write Tests

Include:

- Unit tests
- Integration tests
- End-to-end tests (when appropriate)

Testing reduces production issues and increases confidence during changes.

7. Review Code

Code reviews help:

- Catch bugs early
- Improve code quality
- Share knowledge across the team
- Maintain consistency

Never treat reviews as a formality.

8. Consider Security from Day One

Common areas:

- Authentication
- Authorization
- Input validation
- Encryption
- Secrets management

Fixing security issues after release is expensive.

9. Think About Performance

Ask:

- Will this scale to thousands of users?
- Are database queries optimized?
- Is caching needed?

Avoid premature optimization, but don't ignore obvious bottlenecks.

10. Handle Errors Properly

Don't just make features work:

- Log errors clearly.
- Show user-friendly messages.
- Implement monitoring and alerts.

Production systems fail; your job is to make failures manageable.

11. Communicate Frequently

Strong communication is critical:

- Share blockers early.
- Give status updates.
- Ask questions when requirements are unclear.

Many project failures are communication failures, not technical failures.

12. Use CI/CD

Automate:

- Builds
- Tests
- Deployments

Tools such as [GitHub Actions](#) , [GitLab CI/CD](#) , and [Jenkins](#) help reduce deployment errors.

13. Monitor Production

After deployment:

- Track logs
- Monitor errors
- Measure performance
- Watch business metrics

Useful tools include [Datadog](#) , [New Relic](#) , and [Grafana](#) .

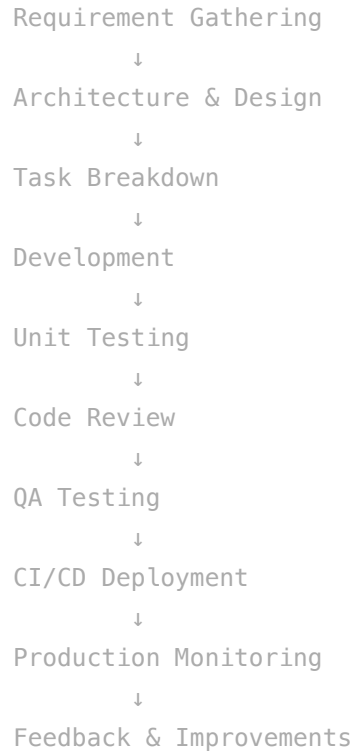
14. Document Important Things

Document:

- Architecture decisions
- API specifications
- Deployment process
- Troubleshooting steps

Good documentation speeds up onboarding and maintenance.

A Typical Real-World Development Flow



What Senior Engineers Usually Focus On

- Understanding the business impact
- Writing maintainable code
- Designing scalable systems
- Reducing technical debt
- Automating repetitive work
- Preventing future problems, not just fixing current ones

A useful rule is: **"First make it work, then make it clean, then make it scalable."** This helps teams deliver value while keeping technical quality under control.

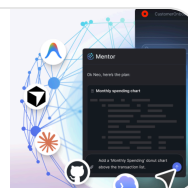


OutSystems
Sponsored



Your Legacy Code Isn't Going Anywhere

Don't rebuild from scratch. Extend core systems with AI on one unified platform.



what are the step to start

If you're starting a real-world software project from scratch, a practical sequence is:

Step 1: Understand the Business Requirement

Meet with stakeholders or clients and answer:

- What problem are we solving?
- Who will use the system?
- What are the main features?
- What are the success criteria?

Output: Requirement document or user stories.

Step 2: Define Scope

Separate:

- Must-have features (MVP)
- Nice-to-have features
- Future enhancements

This prevents scope creep.

Output: Prioritized feature list.

Step 3: Choose the Technology Stack

Decide on:

- Frontend (e.g., [React](#) , [Angular](#) , [Vue.js](#))
- Backend (e.g., [Node.js](#) , [Spring](#) , [Django](#))
- Database (e.g., [PostgreSQL](#) , [MySQL](#))
- Cloud platform if needed

Choose technologies based on project requirements, not popularity.

Step 4: Design the Architecture

Create:

- High-level architecture diagram
- Database schema
- API design
- Module breakdown

Output: Technical design document.

Step 5: Set Up the Project

Create:

- Git repository
- Branching strategy
- Development environment
- CI/CD pipeline
- Coding standards

Output: Project skeleton ready for development.

Step 6: Create Tasks and Sprint Plan

Break work into:

- User stories
- Development tasks
- Testing tasks

Estimate effort and assign priorities.

Output: Sprint backlog.

Step 7: Develop the MVP

Start with core functionality:

1. Authentication
2. Main business workflow
3. Basic UI

4. Database integration

Avoid building advanced features first.

Step 8: Test Continuously

Perform:

- Unit testing
- Integration testing
- Manual testing
- Security checks

Fix issues early.

Step 9: Code Review and Merge

Before merging:

- Review code quality
- Verify requirements
- Ensure tests pass

Step 10: Deploy and Monitor

Deploy to:

- Development
- Staging
- Production

Monitor:

- Errors
- Performance
- User feedback

Simple Example (Employee Management System)

1. Gather requirements
↓
2. Define features
↓
3. Design database
↓
4. Design APIs
↓
5. Setup Git repository
↓
6. Create project structure
↓
7. Build Login Module
↓
8. Build Employee CRUD
↓
9. Test application
↓
10. Deploy and monitor

For a software engineer joining a company project, the very first things to do are usually:

1. Understand the requirement.
2. Understand the existing architecture.
3. Set up the project locally.
4. Run the application successfully.
5. Study the database and APIs.
6. Pick a small task/bug fix first.
7. Gradually move to larger features.

This approach helps you become productive without getting overwhelmed by a large codebase.



Akveo Inc
Sponsored



Build Custom Software Your Business

From MVP to enterprise solutions. 70 engineers, excellent delivery, free estimate.



what to start

If you're asking "What should I start with when beginning a software project?", use this order:

1. Understand the Requirement

Before coding, answer:

- What problem are we solving?
- Who are the users?
- What features are required?

2. Create User Stories

Example:

- As a user, I want to log in.
- As an admin, I want to manage users.

3. Design the System

Create:

- Architecture diagram
- Database design
- API design
- UI wireframes (optional)

4. Set Up the Development Environment

- Create repository in Git.
- Configure project structure.
- Set up database.
- Configure local environment.

5. Build the Foundation

Usually start with:

- Authentication/Login
- User management
- Database connectivity

- Common utilities

6. Develop Core Features

Implement the main business functionality first.

7. Test Everything

- Unit tests
- Integration tests
- Manual testing

8. Deploy and Get Feedback

Release an MVP and gather feedback before adding more features.

A Quick Rule



If you're a developer joining an **existing company project**, start with:

1. Running the application locally.
2. Understanding the architecture.
3. Understanding the database.
4. Understanding the APIs.
5. Taking a small bug fix or enhancement first.

That is usually the fastest and safest way to get productive on a real project.



where to start

The answer depends on whether you're starting a **new project** or joining an **existing project**.

If it's a New Software Project

Start here:

1. Requirements

- What problem are you solving?
- Who are the users?
- What features are needed?

2. High-Level Design

- Application architecture
- Database design
- API design

3. Project Setup

- Git repository
- Development environment
- Basic project structure

4. Core Features

- Authentication (if needed)
- Main business workflow
- Database integration

5. Testing and Deployment

If You're Joining an Existing Project

Start in this order:

1. Run the Application Locally

The first goal is to get the project running on your machine.

2. Understand the Project Structure

Look at:

- Frontend code
- Backend code
- Configuration files
- Database scripts

3. Understand the Business Flow

Ask:

- What does the application do?
- Who uses it?
- What are the main features?

4. Study the Architecture

Understand:

- How requests flow through the system
- API structure
- Database relationships
- External integrations

5. Review Recent Work

Check:

- Recent Git commits
- Open bugs
- Current sprint tasks

6. Pick a Small Task First

Start with:

- A minor bug fix
- A small enhancement
- A simple API change

This helps you learn the codebase safely before working on larger features.

A Common Approach Used by Experienced Developers

Understand Business Requirements

↓

Run Application Locally

↓

Understand Architecture

↓

Understand Database

↓
Understand APIs
↓
Fix Small Bug
↓
Implement Small Feature
↓
Take Larger Tasks

If you're joining a project as a Java, .NET, Python, or frontend developer, tell me your technology stack and I can give a more specific "first week onboarding" roadmap.

